

INDIAN INSTITUTE OF TECHNOLOGY KANPUR

CS641: MODERN CRYPTOLOGY

PROJECT REPORT

**Do not Mix Models:
Revisiting Generic Transforms for
Committing Authenticated Encryption**

Based on: Minematsu & Inoue, Do not Mix Models

Mentor: Prof. Somitra Sanadhya

Mohammed Anas (220654)

Venkatesh Akula (220109)

May 2026

Contents

1	Introduction	2
2	Background	2
2.1	Authenticated Encryption	2
2.2	Committing Authenticated Encryption	3
3	Generic Transforms and Why They Are Attractive	3
4	The Mixed-Model Problem	3
5	Methodology of the Paper	4
6	Transforms Studied in the Paper	4
6.1	AEaH	4
6.2	EtH	5
6.3	HtAE	5
6.4	AEtH	5
6.5	chaSIV	5
6.6	PACT and comPACT	6
7	Main Attack Ideas	6
7.1	Privacy Attack on AEaH	6
7.2	Authenticity Attack on AEaH	7
7.3	Attacks on EtH	7
7.4	Attack on AEtH	8
7.5	Attack on chaSIV	8
7.6	Attacks on PACT and comPACT	9
7.7	Key Recovery via Key Blinding	9
8	Summary of Attacks	10
9	Fixes Proposed in the Paper	10
9.1	Root Cause	10
9.2	Key Separation	10
9.3	AEaH-KS	10
9.4	EtH-KS	11
9.5	AEtH-KS	11
9.6	Security Bounds for AEaH-KS	11
9.7	Fixing chaSIV	11
9.8	HtAE-ST	12
10	Why Random IVs Do Not Solve the Problem	12
11	Discussion and Limitations	12
12	Broader Lesson	12
13	Conclusion	13

1 Introduction

Authenticated encryption is one of the most widely used symmetric-key primitives. It is used in protocols where we want both secrecy and integrity. A normal authenticated encryption scheme takes a key K , nonce N , associated data A , and message M , and produces a ciphertext C , often together with a tag T . Its usual goals are privacy and authenticity. Privacy means that the ciphertext should hide the message, and authenticity means that an attacker should not be able to forge a new valid ciphertext.

However, recent work has shown that ordinary AE security does not always capture everything we need in real applications. One important missing property is *committing security*. Roughly, committing security says that a ciphertext should not be explainable in two different ways. More formally, it should be hard to find two distinct tuples

$$(K, N, A, M) \neq (K', N', A', M')$$

such that

$$\Pi.\mathcal{E}(K, N, A, M) = \Pi.\mathcal{E}(K', N', A', M').$$

This is stronger than ordinary privacy and authenticity because the adversary is now allowed to choose the keys as part of the attack. In ordinary AE security, the key is secret and fixed by the challenger. In committing security, the key itself becomes part of the context that the ciphertext should bind to.

We found this distinction important because many standard AE schemes were not designed with this stronger goal in mind. For example, AES-GCM is secure for the usual AE goals under proper nonce usage, but it is not automatically committing. This motivates a natural research question: can we take any secure AE scheme and transform it into a committing AE scheme in a generic way?

The paper studies exactly this question. It focuses on generic transforms that take a baseline AE scheme Π and a hash function H , and output a new committing AE scheme $\Omega[\Pi, H]$. The problem is that several recent transforms prove security by assuming that H is an ideal random oracle, while Π is only assumed to be secure in the standard model. The authors show that this mixture creates a real gap once the random oracle is replaced by an actual hash function.

2 Background

2.1 Authenticated Encryption

An authenticated encryption scheme is usually written as

$$\Pi = (\Pi.\mathcal{E}, \Pi.\mathcal{D}),$$

where encryption is

$$C \leftarrow \Pi.\mathcal{E}(K, N, A, M),$$

and decryption is

$$M \leftarrow \Pi.\mathcal{D}(K, N, A, C).$$

If the ciphertext is invalid, decryption returns

$$\perp.$$

The inputs have the following meanings:

K = secret key,

N = nonce,

A = associated data,

M = message/plaintext.

The associated data A is not encrypted, but its integrity is protected. A simple example is a packet header: we may not want to hide it, but we do want to detect if someone modifies it.

The two standard AE goals are:

PRIV : ciphertexts should hide plaintexts,

AUTH : attackers should not forge valid ciphertexts.

In our slides, we wrote PRIV as saying that (C, T) is indistinguishable from random, and AUTH as saying that an adversary cannot forge a valid (C, T) . This is the usual black-box security viewpoint: the adversary interacts with encryption and decryption oracles but does not choose or know the secret key.

2.2 Committing Authenticated Encryption

Committing security asks for something different. It says that the ciphertext should uniquely bind to its full encryption context. The strongest notion considered in the paper is *context-committing security*, also called CMT-4.

The condition is that it should be hard to find

$$(K, N, A, M) \neq (K', N', A', M')$$

such that

$$\Pi.\mathcal{E}(K, N, A, M) = \Pi.\mathcal{E}(K', N', A', M').$$

This definition includes all components: key, nonce, associated data, and message. That is why it is called context committing. The ciphertext is not just hiding the message and preventing forgery; it is also committing to the whole context.

We can understand this using a simple analogy. Ordinary AE says: nobody should read or forge the sealed envelope. Committing AE says something more: the same sealed envelope should not later be opened in two different ways. In other words, once a ciphertext is created, it should not be possible to claim that it corresponds to another message under another key.

This matters in attacks like partition oracle attacks and invisible salamanders. In a partition oracle attack, key-commitment failure can help an attacker test key guesses. In an invisible salamander attack, the same encrypted file may decrypt to different meaningful contents under different keys. These examples show why committing security is not just a theoretical curiosity.

3 Generic Transforms and Why They Are Attractive

Since most standard AE schemes were not designed to be committing, one natural approach is to build a *generic transform*. A generic transform takes a baseline AE scheme Π , adds some extra processing, usually involving a hash function H , and produces a new committing AE scheme:

$$\Omega[\Pi, H].$$

The hope is:

$$\Pi \text{ is AE-secure} \implies \Omega[\Pi, H] \text{ is AE-secure and committing.}$$

This is attractive because it avoids designing a new committing AE scheme from scratch. Instead, we can reuse existing AE schemes and wrap them.

The transform is supposed to access Π in a black-box way. That means it only calls $\Pi.\mathcal{E}$ and $\Pi.\mathcal{D}$. It should not depend on the internal structure of Π .

The CTX transform of Chan and Rogaway was an important starting point. CTX uses a hash of the AE context and the baseline AE tag. If the baseline AE produces ciphertext and tag (C, T) , CTX computes

$$T^* = H(K, N, A, T),$$

and outputs

$$(C, T^*).$$

In the random-oracle model, CTX was shown to give context-committing security. However, CTX is not strictly generic because it requires the AE scheme to be tag-based and requires changes to decryption. This motivated later transforms such as those by Chen–Karadžić and Bhattacharjee et al., which try to be more compatible with standard AE APIs.

4 The Mixed-Model Problem

The central issue studied in the paper is the mixed security model.

There are two proof worlds involved:

1. **Standard model:** We assume concrete security properties such as PRF security, PRG security, collision resistance, or PRP security.
2. **Idealized model:** We assume ideal mathematical objects such as random oracles or ideal ciphers.

A random oracle is not a real hash function. It is an ideal public function

$$RO : \{0, 1\}^* \rightarrow \{0, 1\}^n$$

that returns a fresh uniformly random output for each new input, while returning the same output if the same input is queried again.

In many generic transforms, the baseline AE Π is assumed to be secure in the standard model, but the hash H used by the transform is assumed to be a random oracle. This is the mixed model:

standard-model AE + idealized hash.

The proof may be correct inside this model. But in implementation, H must be instantiated by a real hash function such as SHA-2 or SHA-3. Once that happens, the baseline AE Π may also use the same hash internally. This is where the proof no longer matches reality.

We understood the paper’s main warning as follows:

The transform assumes that H behaves like an ideal independent object. But after instantiation, H becomes public code, and the baseline AE can be deliberately designed to use exactly the same code in a correlated way.

This is not a cryptanalytic attack on SHA-3. Rather, it is an attack on the generic proof guarantee. The authors show that a baseline AE can be secure by itself, but when it is wrapped with the transform, the transform leaks a hash value that the AE was using internally.

5 Methodology of the Paper

The authors follow a clear methodology.

First, for each target transform Ω , they instantiate the ideal hash H with a concrete hash function. Then they construct a baseline AE scheme Π that internally uses the same H . This Π is not simply insecure; it is designed to be AE-secure on its own under reasonable standard-model assumptions on H .

Then they compose the two:

$$\Omega[\Pi, H].$$

The attack shows that the composed scheme is broken, even though the baseline Π is secure by itself. This is the important point: the weakness appears because of the interaction between Π and the transform.

The authors classify transforms using two notions:

- **Uninstantiable:** for any instantiation of the ideal primitive, there exists a secure baseline AE such that the transformed scheme is broken.
- **Conditionally uninstantiable:** the attack works for some class of instantiations.

The general attack pattern is:

Design Π using H ,
 Π remains AE-secure alone,
 $\Omega[\Pi, H]$ becomes insecure.

This is why the paper says that a malicious party could hide a backdoor in a baseline AE: the AE passes standalone security checks, but the wrapped construction leaks information.

6 Transforms Studied in the Paper

6.1 AEaH

AEaH stands for AE and Hash. It runs the baseline AE and separately hashes the context:

$$\begin{aligned} C &\leftarrow \Pi.\mathcal{E}(K, N, \varepsilon, M), \\ T &\leftarrow H(K, N, A). \end{aligned}$$

The output is

$$(C, T).$$

Here, the baseline AE is called with empty associated data ε , and the hash tag is supposed to handle A . The transform is simple, but this simplicity is exactly what the attack exploits.

6.2 EtH

EtH stands for Encrypt then Hash. It first encrypts using a plain encryption scheme Γ :

$$C \leftarrow \Gamma.\mathcal{E}(K, N, M),$$

then computes

$$T \leftarrow H(K, N, A, C).$$

The output is again

$$(C, T).$$

Unlike AEaH, the tag here depends on the ciphertext as well.

6.3 HtAE

HtAE stands for Hash then AE. It first hashes the context to derive both an encryption key and a tag:

$$(K_e, T) \leftarrow H(K, N, A).$$

Then it encrypts using the derived key:

$$C \leftarrow \Pi.\mathcal{E}(K_e, N, \varepsilon, M).$$

The paper does not find an instantiation attack against HtAE. Later, the authors show that HtAE can be proven secure in the standard model by assuming H is a PRF and right-half collision resistant.

6.4 AEtH

AEtH stands for AE then Hash. It first runs the baseline AE:

$$C \leftarrow \Pi.\mathcal{E}(K, N, A, M),$$

then hashes the context and the last τ bits of the ciphertext:

$$T \leftarrow H(K, N, A, \text{lsb}_\tau(C)).$$

The output is

$$(C, T).$$

This transform is meant to preserve misuse-resistant AE security when the baseline Π already has it.

6.5 chaSIV

chaSIV is a committing hash-based SIV-like transform. It computes

$$K_e \leftarrow F(K, N),$$

$$T \leftarrow H(K, N, A, M),$$

$$N_e \leftarrow \text{lsb}_r(T),$$

and then encrypts

$$C \leftarrow \Gamma.\mathcal{E}(K_e, N_e, M).$$

The output is

$$(C, T).$$

This construction is already mixed internally because it uses a standard-model PRF F and a random-oracle-style hash H .

6.6 PACT and comPACT

PACT and comPACT come from Bhattacharjee et al. They work by encrypting the baseline AE's tag. If the baseline AE produces

$$(\tilde{C}, \tilde{T}),$$

then PACT computes

$$K^* = H(K, N, A),$$

$$T = E(K^*, \tilde{T}).$$

The final output is

$$(\tilde{C}, T).$$

comPACT is similar, but it passes empty associated data to the baseline AE to reduce repeated processing.

7 Main Attack Ideas

The easiest way to understand the paper is through the AEaH privacy attack. Almost all other attacks follow the same philosophy.

7.1 Privacy Attack on AEaH

The transform outputs

$$T = H(K, N, A).$$

Now suppose the baseline AE uses H in counter mode. For a block message

$$M = M[1] \| M[2] \| \dots \| M[m],$$

define

$$\tilde{C}[i] = H(K, N, \langle i \rangle) \oplus M[i].$$

This is just stream-cipher-style encryption: produce a keystream block and XOR it with the plaintext block.

The attacker chooses

$$A = \langle 1 \rangle.$$

Then AEaH outputs

$$T = H(K, N, \langle 1 \rangle).$$

But this is exactly the keystream used in the first ciphertext block:

$$\tilde{C}[1] = H(K, N, \langle 1 \rangle) \oplus M[1].$$

Therefore,

$$\tilde{C}[1] \oplus T = H(K, N, \langle 1 \rangle) \oplus M[1] \oplus H(K, N, \langle 1 \rangle) = M[1].$$

So the first plaintext block is recovered.

This attack is very clean. The baseline AE is secure by itself because $H(K, N, \langle i \rangle)$ behaves like a pseudorandom keystream if H_K is a PRF. But AEaH leaks the same keystream block as its tag. In our understanding, this one equation captures almost the whole paper:

$$\text{transform tag} = \text{baseline keystream}.$$

Once that happens, privacy is gone.

7.2 Authenticity Attack on AEaH

The AEaH authenticity attack uses a GCM-like baseline AE. The baseline tag is

$$\tilde{T} = \text{GHASH}(L_2, (A, \tilde{C})) \oplus H(K, N, A).$$

AEaH separately outputs

$$T = H(K, N, A).$$

So the attacker can compute

$$\tilde{T} \oplus T = \text{GHASH}(L_2, (A, \tilde{C})).$$

The masking value cancels. Since GHASH is algebraic, exposing raw GHASH outputs on chosen inputs can allow the attacker to recover the GHASH key L_2 , similar to classical attacks on GCM under nonce misuse. Once L_2 is known, the attacker can forge valid ciphertexts.

The important idea here is that the baseline AE masks an internal MAC value using $H(K, N, A)$, but the transform leaks exactly that mask.

7.3 Attacks on EtH

EtH computes

$$T = H(K, N, A, C).$$

The privacy attack is slightly more complicated because C is also part of the hash input. The authors use a two-query trick.

The baseline encryption is defined using a hash-based counter mode:

$$C[i] = \text{msb}_{|M[i]|}(H(K, \text{cst}, N, \langle i - 1 \rangle)) \oplus M[i].$$

In the first query, the attacker sets

$$N = \text{cst}.$$

Then EtH produces

$$T = H(K, \text{cst}, A, C).$$

In the second query, the attacker sets

$$N' = A.$$

They choose an index j such that

$$\langle j - 1 \rangle = C.$$

Then the keystream block in the second query becomes

$$H(K, \text{cst}, A, C),$$

which is exactly the previous tag T . Thus the attacker can predict a ciphertext block in the second query and distinguish real encryption from random.

For authenticity, the attacker learns an internal keystream value

$$H(K, N, \text{cst}, \langle i \rangle) = M[i] \oplus C[i],$$

and then reuses it as a valid tag by choosing

$$A' = \text{cst}, \quad C' = \langle i \rangle.$$

Since EtH verifies

$$T' = H(K, N, A', C'),$$

the forged tag is accepted.

7.4 Attack on AEtH

AEtH computes

$$T = H(K, N, A, \text{lsb}_\tau(C)).$$

The attack uses a SIV-like baseline AE. The baseline computes

$$V = F(K, N, A, M),$$

then encrypts

$$C[i] = M[i] \oplus H(K, N, A, V \oplus \langle i-1 \rangle_\tau).$$

The ciphertext contains V at the end, so

$$\text{lsb}_\tau(C) = V.$$

Therefore AEtH outputs

$$T = H(K, N, A, V).$$

For the first block,

$$V \oplus \langle 0 \rangle_\tau = V,$$

so

$$C[1] \oplus M[1] = H(K, N, A, V) = T.$$

Hence

$$M[1] = C[1] \oplus T.$$

This gives plaintext recovery.

The paper does not find an authenticity attack on AEtH, but the privacy attack is already enough to show that the transform is not safely instantiable.

7.5 Attack on chaSIV

chaSIV computes

$$K_e = F(K, N),$$

$$T = H(K, N, A, M).$$

The attack works when F itself is implemented using H . For example,

$$F(K, N) = \text{msb}_\kappa(H(K, N, \text{cst}_1, \text{cst}_2)).$$

Now the attacker encrypts with

$$A = \text{cst}_1,$$

$$M = \text{cst}_2.$$

Then

$$T = H(K, N, \text{cst}_1, \text{cst}_2).$$

So

$$K_e = \text{msb}_\kappa(T).$$

In other words, the tag directly reveals the encryption key K_e . This gives a complete privacy break.

This attack is called conditional because it requires F to use H . It does not work for every possible independent implementation of F , but it is still meaningful because PRFs are often built from hash functions in practice.

7.6 Attacks on PACT and comPACT

PACT computes

$$\begin{aligned} K^* &= H(K, N, A), \\ T &= E(K^*, \tilde{T}). \end{aligned}$$

For the privacy attack, the baseline AE is designed so that

$$\tilde{C}[i] = M[i] \oplus E(K^*, \tilde{T} \oplus \langle i-1 \rangle_\tau).$$

For $i = 1$,

$$\tilde{C}[1] = M[1] \oplus E(K^*, \tilde{T}).$$

But PACT outputs

$$T = E(K^*, \tilde{T}).$$

Therefore,

$$M[1] = \tilde{C}[1] \oplus T.$$

So the transform leaks the first plaintext block.

For authenticity, the baseline AE chooses

$$\tilde{T} = E^{-1}(K^*, V),$$

where

$$V = \text{GHASH}(L, (A, \tilde{C})).$$

PACT then computes

$$T = E(K^*, \tilde{T}) = E(K^*, E^{-1}(K^*, V)) = V.$$

So PACT directly reveals the GHASH output V . From this, the attacker can recover the GHASH key and forge ciphertexts.

The same idea applies to comPACT with minor modifications.

7.7 Key Recovery via Key Blinding

The paper also shows that some attacks can be upgraded to key recovery using a slightly artificial technique called key blinding.

Instead of defining encryption as

$$\tilde{C}[i] = M[i] \oplus H(K, N, \langle i \rangle),$$

define

$$\tilde{C}[i] = M[i] \oplus H(K, N, \langle i \rangle) \oplus K.$$

By itself, K is hidden because it is masked by the unknown hash output. But if the transform leaks

$$T = H(K, N, \langle 1 \rangle),$$

then the attacker computes

$$K = M[1] \oplus \tilde{C}[1] \oplus T.$$

The authors treat these examples as somewhat artificial, but they make the point stronger: the mixed-model gap can lead not only to privacy or authenticity breaks, but even to key recovery.

8 Summary of Attacks

The attacks can be summarized as follows:

Transform	Privacy	Authenticity	Key Recovery
AEaH	broken	broken	with key blinding
EtH	broken	broken	with key blinding
AEtH	broken	no attack found	with key blinding
chaSIV	conditional break	no attack found	conditional variant
PACT	broken	broken	with key blinding
comPACT	broken	broken	with key blinding

The paper also discusses CTX and CTY. Although they rely on a mixed model, the authors do not find attacks against them. One reason is that CTX and CTY replace the baseline tag rather than exposing it directly. Still, the paper shows that key-separated standard-model variants can be obtained.

9 Fixes Proposed in the Paper

9.1 Root Cause

The root cause is shared computation. The transform uses

$$H(K, \dots),$$

and the baseline AE can also use

$$H(K, \dots)$$

internally. If the attacker chooses inputs so that the two hash inputs match, the transform may leak a keystream block, MAC mask, or encryption key.

We think the paper’s repair idea is very natural once the problem is seen clearly: do not let the AE and the transform use the same effective keyed hash.

9.2 Key Separation

The main fix is Key Separation. Instead of using K everywhere, derive two keys:

$$(K_1, K_2) \leftarrow G(K),$$

where G is a PRG.

Then:

K_1 is used for the baseline AE,

K_2 is used for the transform hash.

This prevents the baseline AE from simulating the transform’s hash. Even if both parts use the same hash algorithm, they use independent derived keys.

A practical advantage is that

$$G(K)$$

can be computed once per key and cached. So the online overhead is small.

9.3 AEaH-KS

Original AEaH computes

$$C \leftarrow \Pi.\mathcal{E}(K, N, \varepsilon, M),$$

$$T \leftarrow H(K, N, A).$$

The fixed version AEaH-KS computes

$$(K_1, K_2) \leftarrow G(K),$$

$$C \leftarrow \Pi.\mathcal{E}(K_1, N, \varepsilon, M),$$

$$T \leftarrow H(K_2, N, A).$$

The old attack no longer works because the ciphertext keystream would involve K_1 , while the tag uses K_2 .

9.4 EtH-KS

EtH-KS similarly computes

$$\begin{aligned}(K_1, K_2) &\leftarrow G(K), \\ C &\leftarrow \Gamma.\mathcal{E}(K_1, N, M), \\ T &\leftarrow H(K_2, N, A, C).\end{aligned}$$

The tag cannot be made equal to an internal keystream block unless the attacker can relate K_1 and K_2 , which should be computationally infeasible if G is secure.

9.5 AEtH-KS

AEtH-KS computes

$$\begin{aligned}(K_1, K_2) &\leftarrow G(K), \\ C &\leftarrow \Pi.\mathcal{E}(K_1, N, A, M), \\ T &\leftarrow H(K_2, N, A, \text{lsb}_\tau(C)).\end{aligned}$$

This protects against the earlier attack where T became the same hash value as the first encryption keystream block.

9.6 Security Bounds for AEaH-KS

The paper gives formal bounds. For privacy,

$$\text{Adv}_{\text{AEaH-KS}}^{\text{priv}}(\mathcal{A}) \leq \text{Adv}_G^{\text{prg}}(\mathcal{A}_G) + \text{Adv}_H^{\text{prf}}(\mathcal{A}_H) + \text{Adv}_\Pi^{\text{priv}}(\mathcal{A}_\Pi).$$

For authenticity,

$$\text{Adv}_{\text{AEaH-KS}}^{\text{auth}}(\mathcal{A}) \leq \text{Adv}_G^{\text{prg}} + \text{Adv}_H^{\text{prf}} + \text{Adv}_\Pi^{\text{auth}} + \frac{q_d}{2^t}.$$

The last term

$$\frac{q_d}{2^t}$$

is the probability of guessing a valid t -bit tag in q_d decryption attempts.

For committing security,

$$\text{Adv}_{\text{AEaH-KS}}^{\text{cmt}}(\mathcal{A}_{\text{cmt}}) \leq \text{Adv}_H^{\text{cr}}(\mathcal{A}_H) + \text{Adv}_G^{\text{rcr}}(\mathcal{A}_G).$$

The proof uses a hybrid argument. First, replace $G(K)$ by two independent random keys. Then replace H_{K_2} by a uniform random function. At that point, the transform tag becomes independent of the baseline AE, and the remaining security reduces to the security of Π .

9.7 Fixing chaSIV

For chaSIV, the paper proposes nonce-based key separation. Instead of

$$K_e = F(K, N)$$

and

$$T = H(K, N, A, M),$$

the fixed version computes

$$\begin{aligned}K_e &= F(K, 0\|N), \\ K_h &= F(K, 1\|N), \\ T &= H(K_h, A, M).\end{aligned}$$

The prefix bits 0 and 1 provide domain separation. The nonce N is removed from the hash input because it is already included in K_h . This prevents the earlier attack where T directly revealed $F(K, N)$.

9.8 HtAE-ST

HtAE is the only CK25 transform for which the paper does not find an instantiation attack. Instead of changing its algorithm, the authors change the assumptions on H . In HtAE-ST, H is assumed to be a PRF and right-half collision resistant.

The transform still computes

$$(K_e, T) \leftarrow H(K, N, A),$$

then encrypts using K_e .

The paper shows that this can be proven secure in the standard model. This is a useful positive result because it shows that not every mixed-model construction is automatically broken; the structure of the transform matters.

10 Why Random IVs Do Not Solve the Problem

One natural idea is to add a fresh random IV to the transform’s hash input. At first, this looks like it should prevent the baseline AE from predicting or matching the transform’s hash call.

But the paper explains why this does not solve the issue. If the receiver has to recompute the hash during decryption, the IV must be included in the ciphertext. Then the adversary sees it. In a later query, the adversary can feed that IV into the baseline AE as part of the nonce or associated data. So the baseline AE can still reproduce the transform’s hash computation.

The lesson is:

public randomness is not the same as secret independence.

Key separation works because K_2 is secret and computationally independent of K_1 , not merely because it adds another public value.

11 Discussion and Limitations

It is important not to overstate the result. The paper does not directly break AES-GCM, ChaCha20-Poly1305, or Ascon in ordinary deployment. The attacks are against generic transform guarantees. The baseline AEs are crafted to exploit the transform.

However, the attacks are still meaningful. A generic transform claims to work for any secure baseline AE. If a secure baseline AE can be constructed such that the transform breaks it, the generic security claim fails.

Another important point is that many of the constructed baseline AEs are not completely unnatural. They use familiar ideas like counter-mode encryption, Enc-then-MAC, and GCM-like universal hashing. This makes the result more convincing than a purely artificial impossibility example.

The paper also does not say that random-oracle proofs are useless. In our understanding, the more accurate message is:

Random-oracle proofs are useful, but when a generic transform combines a standard-model baseline AE with an idealized hash assumption, we must be careful about what happens after instantiation.

12 Broader Lesson

The broader lesson is the title of the paper itself: *Do not mix models*.

The standard model abstracts security properties of real primitives. For example, we may assume a function is a PRF or a hash is collision resistant. The idealized model replaces a primitive by a perfect object like a random oracle or an ideal cipher.

Both models are useful. The danger appears when one component is proved secure in the standard model and another component is proved using an idealized primitive, but in implementation both components rely on the same concrete algorithm.

In this paper, the transform treats H as an ideal random oracle. But after instantiation, say $H = \text{SHA3}$ or something similar. Then the baseline AE may also use SHA-3 internally. The proof did not model this dependency, and the attacks exploit exactly that.

The final conclusion in our slides captures the paper well: generic transforms for committing AE that mix standard-model and idealized assumptions can become vulnerable when instantiated, even if each component is individually secure; key separation gives lightweight standard-model repairs; and AEaH-KS and AEtH-KS achieve strict genericity, CMT-4 security, no nonce-based rekeying, and standard-model security together.

13 Conclusion

In this report, we studied Minematsu and Inoue’s work on generic transforms for committing authenticated encryption. We started from ordinary AE and saw why committing security is a stronger and practically relevant property. We then looked at the idea of generic transforms, where a secure baseline AE is wrapped using a hash function to make it committing.

The main insight of the paper is that several recent transforms rely on a mixed model: the baseline AE is standard-model secure, but the transform’s hash is treated as a random oracle. Once the random oracle is instantiated by a concrete hash function, the baseline AE may use the same hash internally. This creates correlations that the proof does not capture.

The attacks follow a common pattern. The baseline AE uses the concrete hash internally as a keystream, mask, MAC component, or key-derivation component. The transform then outputs a hash value that coincides with that internal value. This leads to privacy breaks, authenticity breaks, plaintext recovery, and in some variants, key recovery.

The cleanest example is AEaH:

$$T = H(K, N, A).$$

If the attacker chooses

$$A = \langle 1 \rangle,$$

and the baseline AE encrypts using

$$\tilde{C}[1] = H(K, N, \langle 1 \rangle) \oplus M[1],$$

then

$$\tilde{C}[1] \oplus T = M[1].$$

This one equation shows the whole problem very clearly.

The paper’s main repair is key separation:

$$(K_1, K_2) \leftarrow G(K).$$

Use K_1 for the baseline AE and K_2 for the transform hash. This prevents the baseline AE from simulating the transform’s hash value. The fixed schemes are lightweight and come with standard-model proofs.

Overall, we understood the paper as a warning about composability and proof models. A construction may be secure in an idealized proof, but if the ideal primitive is later instantiated in a way that interacts with another component, the proof may no longer justify the real construction. The safest way forward, at least for these transforms, is to keep the assumptions entirely in the standard model or to enforce strong separation between components.

References

- [1] Kazuhiko Minematsu and Akiko Inoue. *Do not Mix Models: Revisiting Generic Transforms for Committing Authenticated Encryption*. IACR ePrint.
- [2] T.-H. Chan and P. Rogaway. Generic transforms for committing authenticated encryption. ESORICS, 2022.
- [3] M. Bellare and V. T. Hoang. Committing authenticated encryption and generic transforms. 2024.
- [4] S. Chen and V. Karadžić. Generic transforms for committing authenticated encryption. EUROCRYPT, 2025.
- [5] Bhattacharjee et al. PACT and comPACT transforms for committing authenticated encryption. ePrint, 2024.